

Herramientas CASE y Metodologia Agil

Exploramos las herramientas CASE, el marco de trabajo Agil, el proceso iterativo de desarrollo y las etapas de planeacion, liberacion, produccion y mantenimiento.

CASE

Agil

Planear

Iterar

Produccion

Mantenimiento

PROFESOR

Carlos Godoy

ALUMNOS

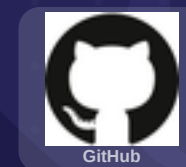
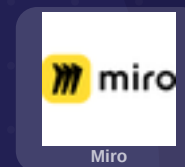
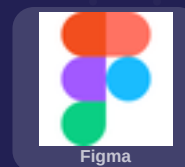
Cardozo Mariano

Manavella Lautaro

Picone Ian

Uso de Herramientas CASE

Las herramientas CASE (Computer-Aided Software Engineering) automatizan y asisten las actividades del ciclo de vida del software, desde el modelado hasta la generacion de codigo, mejorando la productividad y reduciendo errores.



Que son las herramientas CASE

CASE son aplicaciones que asisten a los desarrolladores en las actividades del SDLC: modelado de procesos con UML, diseno de bases de datos, generacion automatica de codigo, documentacion y pruebas. Reducen el esfuerzo manual y minimizan errores en las fases criticas del desarrollo.

CASE de alto nivel (Upper CASE)

Se enfocan en las fases de analisis y diseno. Permiten crear diagramas de flujo de datos, modelos entidad-relacion, casos de uso y prototipos de interfaz. Herramientas como Figma y Miro son ejemplos modernos para el modelado colaborativo en tiempo real.

CASE de bajo nivel (Lower CASE)

Se centran en la implementacion: generacion automatica de codigo desde modelos, refactorizacion asistida, depuracion y pruebas automatizadas. GitHub Copilot y herramientas de CI/CD como Jenkins representan esta categoria en el desarrollo moderno.

Beneficios estrategicos del uso de CASE

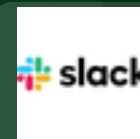
Reducen el tiempo de desarrollo hasta un 40%, mejoran la consistencia entre el diseno y la implementacion, facilitan la colaboracion en equipos distribuidos y generan documentacion actualizada automaticamente, disminuyendo la deuda tecnica acumulada.

La Metodologia Agil

Agil es un conjunto de principios y valores para el desarrollo iterativo e incremental de software. Prioriza la colaboracion, la adaptacion al cambio y la entrega continua de valor sobre la documentacion exhaustiva y los planes rigidos.



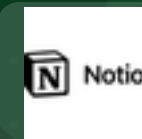
Jira



Slack



Trello



Notion

Manifiesto Agil: los 4 valores fundamentales

El Manifiesto Agil (2001) establece: individuos e interacciones sobre procesos y herramientas; software funcionando sobre documentacion exhaustiva; colaboracion con el cliente sobre negociacion contractual; y respuesta al cambio sobre seguir un plan rigido establecido al inicio.

Scrum: el framework agil mas adoptado

Scrum organiza el trabajo en Sprints de 1-4 semanas. Roles clave: Product Owner (define que se hace), Scrum Master (facilita el proceso) y Development Team (ejecuta). Ceremonias: Sprint Planning, Daily Standup, Sprint Review y Retrospectiva. El Product Backlog prioriza el trabajo pendiente.

Kanban: flujo continuo y visualizacion

Kanban visualiza el trabajo en columnas (Por hacer, En progreso, Hecho) limitando el trabajo en curso (WIP limits). A diferencia de Scrum no tiene iteraciones fijas; el flujo es continuo. Herramientas como Trello, Jira y Linear lo implementan digitalmente para equipos distribuidos.

Principios Agil en la practica diaria

Los equipos agiles trabajan en ciclos cortos, integran codigo frecuentemente (CI/CD), realizan reuniones diarias de 15 minutos para sincronizarse, revisan el progreso con el cliente cada sprint y mejoran continuamente sus procesos mediante retrospectivas honestas y accionables.

Proceso de Desarrollo para un Proyecto Agil

El desarrollo agil es iterativo e incremental: cada iteracion entrega software funcionando. El equipo trabaja en ciclos cortos donde define, construye, prueba y entrega funcionalidades priorizadas por el negocio.



GitHub



Jenkins



SonarQube



Postman

Definition of Ready y Definition of Done

Una historia de usuario esta lista (Ready) cuando tiene criterios de aceptacion claros, estimacion acordada y dependencias resueltas. Esta hecha (Done) cuando pasa todas las pruebas unitarias e integracion, es revisada por pares en GitHub, supera el analisis de calidad en SonarQube y es aprobada por el Product Owner.

Integracion y entrega continua (CI/CD)

Jenkins o GitHub Actions ejecutan automaticamente pruebas unitarias, analisis de codigo estatico con SonarQube, construccion de artefactos y despliegue en ambientes de desarrollo y staging cada vez que un desarrollador hace un push al repositorio de Git.

Pruebas automatizadas en el proceso agil

El desarrollo agil requiere una suite robusta de pruebas: unitarias (Jest, Pytest), de integracion (Postman para APIs REST), de aceptacion (Selenium, Cypress) y de rendimiento (JMeter). La cobertura de codigo se mantiene por encima del 80% para garantizar la calidad en cada sprint.

Roles y responsabilidades del equipo agil

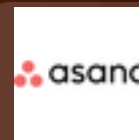
En un equipo agil cada miembro tiene responsabilidades claras: el Product Owner prioriza el backlog con criterio de negocio, el Scrum Master elimina impedimentos, los desarrolladores se auto-organizan para cumplir el sprint goal y el QA define los criterios de calidad desde el inicio de cada historia.

Planeacion del Proyecto Agil

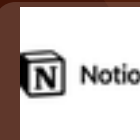
La planeacion agil es adaptativa y multinivel: desde la vision del producto hasta la planificacion diaria de tareas. Se define el backlog priorizado, se estima con Story Points y se organiza el trabajo en sprints con capacidad realista del equipo.



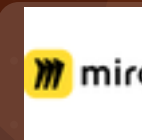
Jira



Asana



Notion



Miro

Vision del producto y Product Roadmap

La planeacion comienza con la vision del producto: que problema resuelve y para quien. El Product Roadmap define los grandes objetivos por trimestre. Luego el Product Backlog desglosa esa vision en epicas, historias de usuario y tareas concretas, todo gestionado en Jira o Notion con prioridad clara.

Estimacion con Story Points y Planning Poker

El equipo estima el esfuerzo relativo de cada historia usando Story Points en una escala de Fibonacci (1, 2, 3, 5, 8, 13, 21). El Planning Poker es la tecnica mas usada: cada miembro vota simultaneamente para evitar la influencia de opinion y llegar a un consenso basado en la experiencia colectiva del equipo.

Sprint Planning: comprometer trabajo realista

Al inicio de cada sprint el equipo selecciona historias del backlog segun su velocidad historica (Story Points completados por sprint). Define el Sprint Goal, desglosa las historias en tareas tecnicas en Jira y confirma que cada item tiene criterios de aceptacion claros antes de comenzar el desarrollo.

Metricas de planeacion: Velocity y Burndown

La Velocity mide cuantos Story Points completa el equipo por sprint, permitiendo predecir fechas de entrega con mayor precision. El Burndown Chart muestra el avance diario del sprint: si la linea real esta por encima de la ideal, el equipo necesita ajustar el alcance o intensificar el esfuerzo.

Iteraciones para la Primera Version

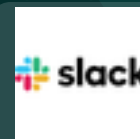
Las iteraciones son ciclos cortos de 1 a 4 semanas donde el equipo entrega incrementos funcionales del producto. La primera version (MVP) se construye a traves de varias iteraciones priorizando las funcionalidades de mayor valor para el usuario.



GitHub



Jira



Slack



Jenkins

MVP: Producto Minimo Viable

El MVP (Minimum Viable Product) es la primera version del sistema con las funcionalidades esenciales para resolver el problema central del usuario. Se construye en 3 a 6 sprints, se lanza a usuarios reales para obtener feedback y se usa ese aprendizaje para priorizar las siguientes iteraciones del producto.

Sprint Review y feedback del cliente

Al final de cada sprint el equipo presenta el incremento funcionando al Product Owner y stakeholders. Se valida si las historias cumplen los criterios de aceptacion, se recoge feedback directo del cliente y se actualiza el backlog con nuevas historias o ajustes de prioridad basados en lo aprendido durante el sprint.

Gestion del backlog entre iteraciones

Entre sprints el Product Owner refina el backlog: anade nuevas historias surgidas del feedback, elimina items que perdieron prioridad, desglosa epicas grandes en historias mas pequenas y re-estima items que han cambiado en alcance. Este refinamiento continuo asegura que el backlog siempre refleje las prioridades actuales del negocio.

Release Planning: coordinar multiples sprints

El Release Planning define cuantos sprints se necesitan para entregar la primera version completa. Se combinan la velocidad del equipo, el backlog priorizado y las fechas comprometidas con el cliente. GitHub Milestones y Jira Releases permiten agrupar historias en versiones y rastrear el progreso hacia cada lanzamiento planificado.

Puesta en Produccion y Mantenimiento

La puesta en produccion es el proceso de desplegar el sistema para usuarios reales. El mantenimiento posterior garantiza la disponibilidad, seguridad y evolucion continua del sistema en funcion de las necesidades cambiantes del negocio.



Estrategias de despliegue en produccion

Docker empaqueta la aplicacion con sus dependencias. Kubernetes orquesta los contenedores en produccion con auto-scaling y alta disponibilidad. Las estrategias mas usadas son Blue-Green Deployment (dos entornos identicos, cambio instantaneo) y Canary Release (el nuevo codigo llega al 5-10% de usuarios primero para validar antes del despliegue total).

Monitoreo y observabilidad del sistema

Datadog, Grafana y AWS CloudWatch monitorizan metricas clave: tiempo de respuesta, tasa de errores (error rate), disponibilidad (uptime) y uso de recursos. Las alertas automaticas notifican al equipo via Slack cuando un metrico supera umbrales criticos, permitiendo respuesta rapida antes de que el impacto llegue a los usuarios.

Mantenimiento correctivo y evolutivo

El mantenimiento correctivo resuelve bugs reportados por usuarios o detectados por el monitoreo automatico. El mantenimiento evolutivo agrega nuevas funcionalidades solicitadas. Ambos tipos usan el mismo proceso Agil: el bug o la nueva feature entra al backlog, se prioriza, se desarrolla en un sprint y se despliega con el mismo pipeline de CI/CD.

Ciclo de mejora continua post-lanzamiento

Cada sprint post-lanzamiento incluye una retrospectiva donde el equipo analiza metricas de uso (cuantos usuarios usan cada feature), feedback de soporte, deuda tecnica acumulada y oportunidades de optimizacion de rendimiento. Este ciclo garantiza que el sistema mejore continuamente en lugar de degradarse con el tiempo.